# LC 1594 – Maximum Non-Negative Product in a Matrix

## Intuition (Must Understand)

1. Multiplication with negatives can flip sign.
2. A very small (negative) value can become very large later.
3. So best answer may come from worst intermediate value.
4. We cannot trust only maximum path.
5. We must track both extremes: max and min.

## Pattern Recognition

1. Grid + path constraint → DP.
2. Multiplication + negatives → track min and max.
3. If operation can flip order → store multiple states.
4. Same pattern appears in max product subarray.

## How to Think

1. Movement: right and down → grid DP.
2. Product instead of sum → be careful.
3. Negative values → sign flips.
4. Need to preserve possibilities.

## Algorithm

1. At each cell store max and min.
2. Use top and left cells.
3. If val >= 0 → normal transition.
4. If val < 0 → swap roles.
5. Apply MOD only at end.

## Clean Solution

```
class Solution:
    def maxProductPath(self, grid):
        MOD = 10**9 + 7
        m, n = len(grid), len(grid[0])
```

```
        max_dp = [[0]*n for _ in range(m)]
        min_dp = [[0]*n for _ in range(m)]

        max_dp[0][0] = min_dp[0][0] = grid[0][0]

        for i in range(1, m):
            max_dp[i][0] = min_dp[i][0] = max_dp[i-1][0] * grid[i][0]

        for j in range(1, n):
            max_dp[0][j] = min_dp[0][j] = max_dp[0][j-1] * grid[0][j]

        for i in range(1, m):
            for j in range(1, n):
                val = grid[i][j]

                if val >= 0:
                    max_dp[i][j] = max(max_dp[i-1][j], max_dp[i][j-1]) * val
                    min_dp[i][j] = min(min_dp[i-1][j], min_dp[i][j-1]) * val
                else:
                    max_dp[i][j] = min(min_dp[i-1][j], min_dp[i][j-1]) * val
                    min_dp[i][j] = max(max_dp[i-1][j], max_dp[i][j-1]) * val

        res = max_dp[-1][-1]
        return res % MOD if res >= 0 else -1
```

## Your Mistakes (Revision)

Mixing maxVal into minVal

```
minVal = min(maxVal, ...)
```

Never mix intermediate max into min.

Not collecting candidates

```
maxVal = ...
minVal = ...
```

Always compute from full candidate set.

MOD applied early

```
val % MOD
```

Breaks sign logic.

Wrong thinking

You optimized too early instead of preserving possibilities.